

01 / かぐやAI

話して、覚えて、少し忘れる。

声と表情で応える、小さなパーソナルAI。



「おかえり。今日はどうだった？」

何気ない会話を重ねながら、
好きなことや、心地よい話し方を覚えていく。
細かい出来事が薄れても、大切なことは残る。



聞く



考える



話す



覚える



忘れる

128 × 128のドット絵 / 5つの状態 / 3つの記憶テーブル
イラストは見た目と動作の案。実装用の128 × 128素材は別途制作する。

02 / 会話のある日

前に話したことが、次につながる。

同じ話を毎回やり直さず、今の気分にも合わせて応える。



ある日

あなた「疲れた日は、短めに話してほしい」

かぐや「うん。今日はゆっくりしよう」



夜の整理

「疲れているときは、短い返事を好む」
という好みを、会話の根拠と一緒に残す。



別の日

あなた「今日はもうくたくた……」

かぐや「お疲れさま。少し休もうか」

「今日は詳しく教えて」と言われたら、その希望を優先する。
覚えた好みで、今の気持ちを決めつけない。

03 / 体と機能

見える姿と、裏側の役割。

イラストの動作を、システムの機能に対応させる。



聞く・話す

Avatar / TypeScript • Canvas2D

テキストと録音を受け取り、文字・声・口パク・表情で返す。



情報を渡す・順番を決める

Controller / Python • FastAPI

入力、記憶、LLMをつなぐ。画面へ今の状態を知らせる。



考える・言葉にする

LLM

入力と必要な記憶を読み、返答を作る。DBへ直接書き込まない。



覚える・整理する

DB / PostgreSQL

会話はraw_memory、知恵はwisdom、性格や接し方はpersonaに保存。

体=Avatar / 神経=Controller / 思考=LLM / 記憶=DB。

人間の仕組みは、役割を説明するためのたとえ。

04 / 会話の流れ

声をかけてから、返事が届くまで。

Controllerが順に進め、アバターが表情で知らせる。



1 聞く

Avatar → Whisper

録音を文字にする



2 残す

raw_memory

ユーザーの発言を保存



3 思い出す

raw / wisdom / persona

今の話に必要な記憶



6 話す

Avatar / VOICEVOX

文字・声・口パクで返す



5 残す

raw_memory

AIの回答を保存



4 考える

LLM

入力と記憶から回答

文字だけでも使える

テキスト入力ならWhisperを省略。音声を使わないときは、回答を表示して完了。音声生成が失敗しても、文字の返答は残す。

05 / 5つの状態

表情を見ると、今していることが分かる。

待機中の瞬きや呼吸は、LLMを呼ばずに動かす。



待っている

idle

瞬き、呼吸、小さな視線移動。



聞いている

listening

録音中。ボタンで録音を確定する。



考えている

thinking

記憶を探し、回答を待っている。



話している

speaking

音声を再生。音量に合わせて口を動かす。



作業している

working

作業機能を追加したときに使う状態。

再生完了・取消・エラーでidleへ戻る。停止操作はすぐに反映する。

遅れて届いた古い回答はrequest_idで見分け、再生しない。

06 / 記憶の流れ

会話を残し、意味をまとめ、少し忘れる。

日中は保存。毎晩は整理。週1回、接し方を見直す。



日中：言葉を残す

raw_memory / 短期記憶

発言者と日時を記録。AIの言葉とユーザーの言葉を分ける。



毎晩：知恵にまとめる

wisdom / 長期記憶

未処理の会話を要約し、同じ話題に統合。根拠はユーザーの発言から取る。



週1回：接し方に反映する

persona / 性格・応答方針

何度か確かめた傾向を反映。基本性格は固定し、一時的な気分で変えない。



細部は、少しずつ薄れていく

古く使わない記憶は、検索順位を下げる。

7日を超えた処理済みrawは削除する。

未処理分は残し、次の整理で回収する。

この「学習」はDBと応答方針の更新。LLMの重みを再学習するものではない。

画面・音声・記憶をつなぐ。

部品間はHTTP / WebSocket。PostgreSQLの専用接続は記憶APIの内側に置く。



Controllerが通信の窓口になる。
重い音声認識は別プロセスへ渡し、
画面の反応を止めない。

経路	方式	用途
Avatar ↔ Controller	WebSocket	入力・返答・状態通知
Avatar ↔ Controller	HTTP	録音送信・音声取得
Controller ↔ LLM	HTTP(S)	会話・要約・更新候補
Controller ↔ 音声API	HTTP	Whisper / VOICEVOX
Controller ↔ 記憶API	HTTP	保存・検索・更新
記憶API ↔ PostgreSQL	DB接続	API内部だけで使用 [1]

FastAPIは1 workerから開始。非同期HTTPで待機し、Whisper推論は専用プロセスで実行する [2]。
VOICEVOXは独立エンジン呼び出す [4]。LLM待機中にDBトランザクションを開いたままにしない。

同一アプリ内のHTTPにも往復の負荷はある。統一方針を保ち、まとめて呼ぶ。
端末内APIはloopbackへ限定。Originとトークンを検証する。
外部LLM利用時は、選んだ記憶が端末外へ送られることを表示する。

08 / 記憶の管理

間違いを覚えたら、直せるようにする。

3テーブルのまま、根拠・訂正・再実行を扱う。



AIが言ったことを、ユーザーの事実にしない。
明示された内容と推測を分け、
同じ発言を何度も根拠に数えない。

テーブル	主な項目（設計案）
raw_memory	id, turn_id, role, content, created_at, processed_at, delivery_status
wisdom	id, topic_key, summary, kind, support_level, importance, evidence, last_seen_at, last_used_at, revision, updated_at
persona	key, value, locked, source_wisdom_ids, revision, updated_at, previous_value

整理は「対象IDとrevisionを控える → LLMで候補作成 → 検証して確定」。wisdom更新とrawの処理済み設定を同じトランザクションで保存する。競合時は再取得。失敗時は未処理を残す。



「それは違う」「忘れて」と言われたら、
元の会話だけでなく派生したwisdom・personaも直す。
rawの期限削除後に残る根拠要旨も、削除対象に含める。

support_levelは未確認・明示・反復などの区分で、正しさの確率ではない。
検索は直近raw・少数persona・話題キーから始め、検索漏れを見て改善する。

09 / 作る順番

小さく動かして、使い心地を確かめる。

最初から常時録音や自律作業まで広げず、会話と記憶を先に整える。

順番	作るもの	確かめること
1	文字の会話+raw	再送で重複しない。切断後に回答を戻せる。
2	wisdom+忘却	訂正が効く。整理失敗で会話を失わない。
3	音声+5状態	録音・再生を止められる。文字でも使える。
4	persona更新	一時的な気分で性格が急変しない。



軽さと費用は、実際に測る

文字が出るまで / 声が出るまでの時間、
待機時のCPU・RAM、推論時のVRAM、
1日のトークン量を記録する。

Whisperは多言語base / smallから日本語で比較する。公式の必要VRAM目安は約1GB / 2GB。速度は端末や言語で変わる [3]。音声利用中はモデルを保持し、利用を終えたら解放する。

会話は原則1往復1回のLLM呼び出し。整理は新しい根拠があるときだけ。

記憶の入力量・回答長・日次費用に上限を設ける。

性能と費用は未測定。ローカル実行でも電力とメモリは使う。

10 / 補足

見た目と説明を、同じ動きでそろえる。

今回の変更と、技術仕様を確認した資料。



音声入力



回答生成



知恵化



接し方

絵は飾りとして置くだけでなく、機能や処理の横に配置した。同じ動作は同じ絵で表し、ページを移っても役割を追えるようにした。説明文は短くし、会話例から技術の話へ進む構成にした。

参照資料

[1] PostgreSQL：通信プロトコル

[2] FastAPI：非同期処理と並列処理

[3] Whisper：モデルと必要メモリ

[4] VOICEVOX Engine：音声合成API

ビジュアル：『超かぐや姫！』のかぐやをイメージした、資料用の生成イラスト。
髪色とカジュアルな服装を参考に、紫を差し色にしたドット絵調で統一。

イメージ参照：KADOKAWAの関連書籍紹介・キャラクター設定画

技術資料確認日：2026年9月11日。本文の運用条件は設計案。

掲載イラストは動作の説明用。アプリの画面、実装、性能を検証したものではない。